University of South Florida

Bellini College of AI Cybersecurity and Computing

Microsoft Capstone Detailed Design Document

CIS 4910 Senior Project Fall 2026

Microsoft 2 Team 3

Bruno Rocha • Kapil Srikanth • Ross Volenec • Joseph Bressani

Sponsors: Chad Hage and Brad Lawrence

Revision 1.5 proposed • October 15 2026

Purpose of this architecture

This document defines the design of a repeatable AI coding workflow evaluation system. It connects requirements to components, interfaces, data, deployment boundaries and verification evidence. Native-tool pilots establish practical workflow evidence; a separate controlled harness is designed to enforce common execution conditions. A documentation-only feature matrix records product capabilities without claiming measured effectiveness.

The proposed design uses a local modular application, isolated execution and evaluation environments, immutable evidence records, and a static reporting site. The design emphasizes independent evaluation, reproducible inputs, explicit limits and honest interpretation of results.

Approval state: revision prepared for team review. New contracts, schema refinements and architecture dispositions are proposals until accepted. Existing benchmark evidence is preserved. This document does not certify the implementation or replace SRS approval.

Document control and reading guide

Revision	Change	Review state
1.0–1.4	Original DDD and October 15 additions, as recorded in the supplied PDF.	Historical attribution retained in source PDF [1].
1.5 proposed	Reconciled views; precise interfaces and data contracts; complete feature coverage; corrected comparison and isolation claims; expanded risks and validation.	AI-assisted revision; team technical review pending.

Requirements baseline: repository SRS working source at commit 9bd7f67aff60f6a2d701e62d7bdb95ed32424d60 [2], labeled revision 1.1 with draft Track A additions. The course requires SRS V2. Final conformance must be checked against the approved V2 before submission; the open decision register identifies the known reconciliation items.

Status vocabulary: “proposed” describes this target design; “reported” describes supplied document or historical evidence claims; “verified” requires a specific inspected artifact or recorded test. A design mapping is not a passing test. Unless explicitly identified as historical evidence, diagrams and contracts depict the proposed target.

The document uses the SRS feature IDs F1–F6, modules M01–M13 and interfaces I01–I09. MATRIX:F01–F18 identify a different catalog. Pilot task 003 is not the SRS test-plan category T3. DDD constraints C-1–C-9 are not SRS constraints with the same numbers.

Contents

Sections	Pages
1–4 Scope framework styles and logical view	3–6
5–7 Modules and physical deployment	7–9
8–10 Behavioral workflows and failure semantics	10–12
11–13 Data architecture and contracts	13–15
14–16 Interfaces and Track A	16–18
17–18 Testbed development and publication	19–20
19–21 Scoring performance and operations	21–23
22–24 Decisions and reconciliation	24–26
25–27 Public impacts risks and verification	27–29
Appendix A Requirements traceability	30–34
Appendices B–D Evidence disclosure references	35–37

Use the Word navigation pane for individual headings. Figures describe target workflows; bracketed citations resolve in Appendix D. Full evidence commits appear in Appendix B.

1 Scope and comparison design

The sponsor needs reproducible evidence to support tool adoption. The four pilot tools are Antigravity, Claude Code, Codex and Copilot CLI. Their available configurations differ in model, execution mode, approvals and environment. Native pilots therefore compare complete configured workflows; they do not isolate a causal harness effect or establish a general tool winner.

Approach	Question and output	Boundary
A documentation review	What official sources establish about a defined tool profile. Source-backed feature cells, no scores or rankings.	SRS F6 / Track A; M11. No installation or live testing.
B native-tool pilots	How the recorded tool/model workflow handles a shared task. Raw correctness, timing, approvals, corrections and scope evidence.	Empirical pilot evidence, separate from controlled scoring. Vendor controls recorded rather than assumed.
C controlled harness	How eligible adapters perform with a fixed model and enforced common policy. Validated records and versioned scorecards.	SRS F1–F5 / controlled Track B target. Admission requires capability and isolation checks.

Approaches B and C are empirical workflows; their results are not pooled. “Approach A” is the documentation-only Track A. These labels disambiguate the attached DDD’s three approaches from the SRS’s two tracks. Track A has no evaluator execution or model profile.

Scope includes the JavaScript todo testbed, sandboxed candidate execution, orchestration, independent acceptance and security/license checks, telemetry, reporting and the documented feature matrix. It excludes model training, production deployment of generated code and a hosted multi-tenant harness.

Approach B uses shared tasks and evaluators to establish a useful process before full harness integration. Shared tasks control task difficulty, not all other variables. Future controlled comparisons must demonstrate that every included tool can use the specified model and execution route. Unsupported tools remain in B rather than being represented by a generic model wrapper.

Comparison identity

A comparison group fixes task revision, baseline, evaluator, metric contract, environment class and comparison method. Each run also retains its tool/model profile. In B, profile differences are the disclosed treatment being compared. In C, the model and policy are fixed while the adapter varies. A model update creates a new campaign revision; it never silently joins an earlier group.

2 Framework stakeholders and constraints

The architecture description follows the stakeholder–concern–viewpoint–view structure of ISO/IEC/IEEE 42010 [5]. It uses the 4+1 organization [6] with an additional data view. This is an architectural organization, not a claim of certified standards conformance.

Stakeholder and concern	View and design response
Sponsors: useful, fair adoption evidence	Scope, scenarios and score eligibility; separate A/B/C results and disclose configuration differences.
Developers: implementable boundaries	Logical modules, typed interfaces, schema contracts and ADRs.
Operators: safe reproducible runs	Physical deployment, lifecycle, budget enforcement, recovery and evidence retention.
Readers: interpretable results	Data provenance, release validation, model metadata, accessibility and uncertainty.
Professor and peers: rigor and traceability	Clause-level mapping, requirements baseline, quantitative targets and validation references.
University and sponsor: exposure and cost	Security boundaries, redaction, license gating, resource limits and spend controls.

Constraint	Design response
C-1 local execution	One operator host; no persistent harness application service.
C-2 JavaScript testbed	Node/Express application, browser client, in-memory state; no external database.
C-3 sponsor budget	Campaign requires an explicitly approved spend cap and price table; $100 versus SRS “100 × 4 credits” is unresolved.
C-4 semester schedule	Prioritize repeatable evaluation; defer distributed orchestration.
C-5 no production deployment	Generated application code runs only for evaluation and demonstrations.
C-6 noninteractive control	Applies to C adapter admission; B may be interactive.
C-7 data and policy obligations	Minimize data, separate credentials, redact public evidence and review institutional requirements.
C-8 weekly sponsor demonstration	Reproducible demo from preserved candidate; no need to regenerate live.
C-9 baseline configuration	Record exact settings and deviations; do not infer all runs used defaults.

3 Architectural style and system context

A modular monolith keeps policy, execution, evaluation and publication separable without introducing a service network. In-process contracts reduce deployment overhead for a sequential campaign. The execution pipeline provides explicit stage boundaries; adapters isolate provider and tool-specific behavior. The static site consumes reviewed data and cannot launch a run.

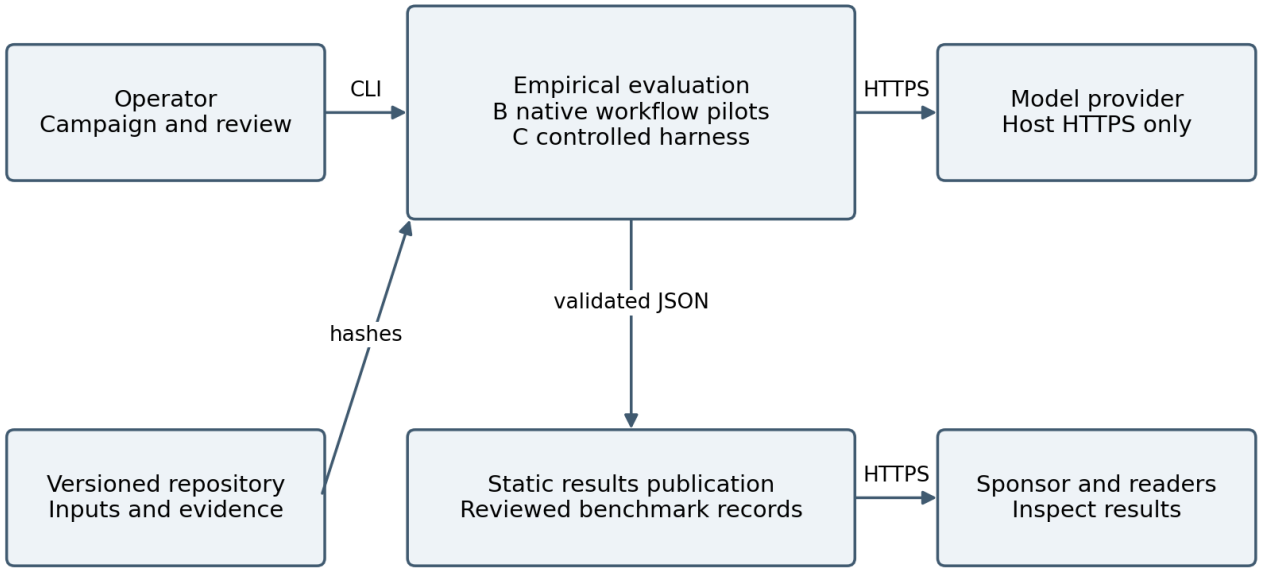


Figure 1. System context. All arrows describe target interactions; Approach A remains documentation-only.

The logical view uses UML-style component boxes and directed dependencies. Physical nodes identify trust boundaries. Sequence views identify message order, and the data view identifies entities and cardinalities. Module IDs are consistent across the diagrams and contract tables.

A pipeline failure is handled by the controller and finalizer rather than passed silently to a later stage. The scoring strategy is selected by an immutable metric-contract version. Event collection is append-only but is not the authoritative score; the terminal record is authoritative after validation.

Build versus buy: reuse Docker isolation primitives, Node/Express, Playwright, JSON Schema validation and GitHub hosting/CI. Build the task manifest validator, adapter contract, freeze/provenance checks, score policy and release validator because they encode project-specific requirements. Reusing a tool does not remove the need to validate its configuration.

4 Logical view

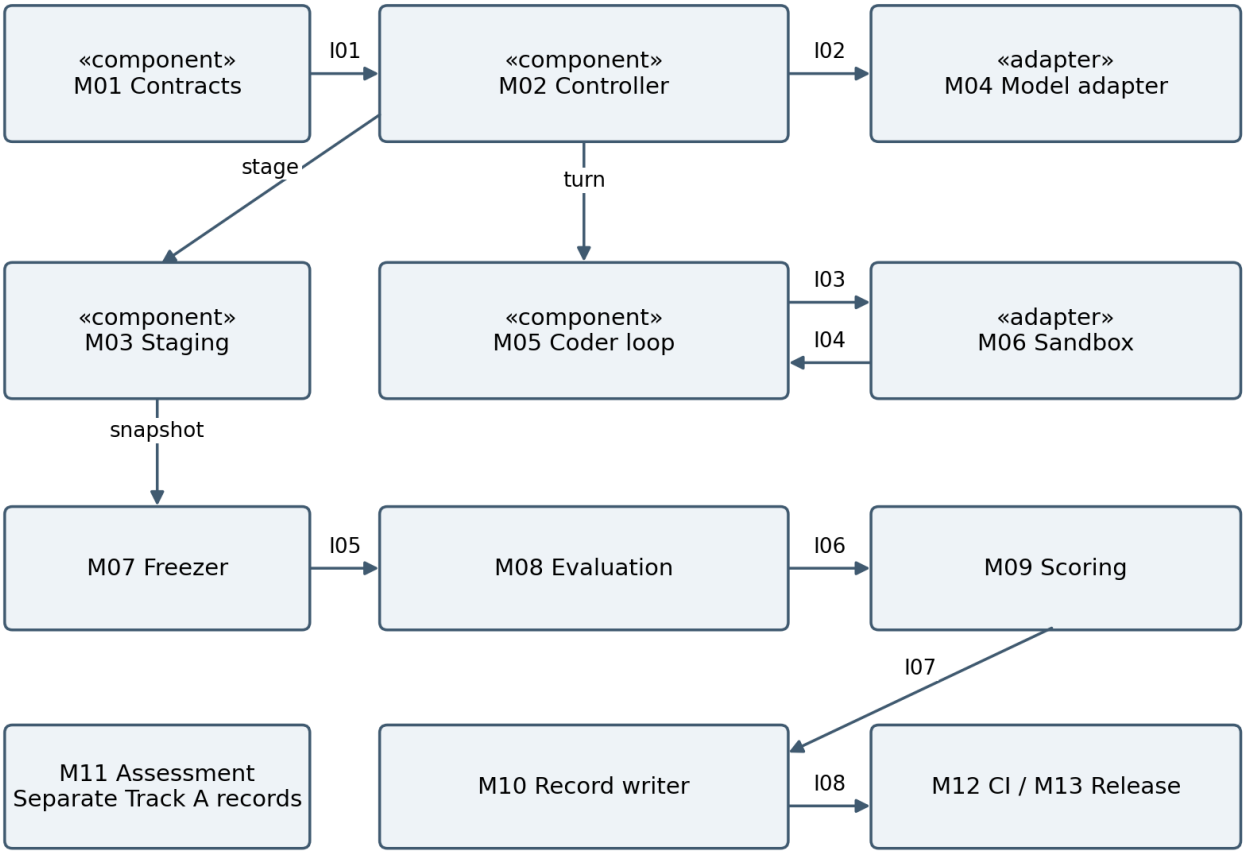


Figure 2. Target logical component view. I09 is adapter admission into M02; it is defined in Section 15.

M02 owns the lifecycle and is the caller of I02. M05 requests a model call through M02; it cannot bypass budget reservation by calling the provider directly. M04 owns the external HTTPS exchange. This distinguishes the internal I02 contract from the provider transport.

M07 stops candidate writers before freezing. M08 executes only evaluator-owned logic against that frozen input. M09 consumes typed results without importing Docker or provider libraries. M10 persists the final record after cleanup status is known. M12 regrades the same candidate revision; M13 publishes only reviewed eligible records.

M11 is independent of the scoring path. It stores documented capability findings and source references. An automatic M11-to-M13 release integration is deferred until the team decides whether Track A belongs on the microsite; its assessment can be delivered independently.

5 Module responsibilities

Module	Responsibility and owned data	Failure boundary
M01 Contracts	Validate campaign, task, profiles, suites and hashes. Own validated immutable inputs.	Reject invalid input before allocating a run ID.
M02 Controller	Allocate run ID; own lifecycle, budget reservations and terminal outcome.	Always invoke finalization; stop scheduling when cap is reached.
M03 Staging	Export pinned seed; copy allowlisted files; attach prepared dependencies.	Reject secrets, unsafe links, changed seed or missing dependencies.
M04 Model adapter	Provider calls, bounded retries, usage completeness, price reference and latency.	Typed provider failure after two retries; unknown usage never becomes zero.
M05 Coder loop	Agent turns, proposed commands, execution feedback and approval requests.	Pause for escalation; propagate cancellation and limit events.
M06 Sandbox	Create isolated execution; enforce quotas, watchdog and teardown.	Fail admission if controls cannot be established; report cleanup failure.
M07 Freezer	Stop writers; copy and hash candidate; compare permitted changes and baseline tests.	Scope violation is explicit; missing candidate is infrastructure failure.

The modules are responsibilities and interface boundaries, not a requirement to create thirteen processes. The host controller coordinates them in one local application. Only untrusted execution and evaluation run in isolated operating-system processes/containers.

Cohesion and dependency control

Each module owns one reason to change: policy in M02, provider behavior in M04, isolation in M06, evaluator implementation in M08, scoring policy in M09 and presentation in M13. Dependencies pass immutable values. Constructors receive adapters rather than importing global clients, avoiding the current prototype's global Docker patch coupling reported in [1].

6 Evaluation records and publication modules

Module	Responsibility and owned data	Failure boundary
M08 Evaluation	Run acceptance suites and secret/license/quality scans; create GraderResult values.	Missing or malformed required report is an infrastructure error, never a passing result.
M09 Scoring	Apply the selected metric contract, overrides and aggregation eligibility.	Return unscored for missing required evidence; keep B and A outside weighted scoring.
M10 Record writer	Construct, validate and atomically persist the terminal record.	Block export if persistence fails; retain bounded recovery journal.
M11 Assessment	Maintain feature catalog references, profile definitions, cells, sources and independent review.	Unknown documentation is a valid status; unsupported claims fail review.
M12 Export and referee	Apply patch onto trusted history; regrade with trusted evaluator; produce bounded CI report.	Reject unsafe patch paths, grader changes, failed required checks or mismatched hashes.
M13 Release builder	Validate eligible records, compute compatible aggregates and publish immutable release.	Any failed check retains the prior release; rollback selects an earlier release hash.

Design principles

Single responsibility separates execution from grading and scoring. Dependency inversion keeps M02 and M09 dependent on contracts, not vendor SDKs. Interface segregation makes capability reporting separate from execution and usage reporting. Open/closed extension adds adapters or evaluators without changing score semantics. Substitutability is tested by the same adapter contract fixtures; an adapter with weaker isolation cannot claim C compatibility.

A thin AutoGen integration may implement M05/M06, but vendor-native workflows require their own adapters. Replacing a vendor harness with an AutoGen agent changes the system being evaluated and must be labeled as a different configuration.

Testbed boundary

The application is an evaluation target, not a controller module. M03 supplies its pinned starting state; M08 verifies its external API/UI contracts. It has no access to records, provider credentials or publication credentials.

7 Physical deployment and trust boundaries

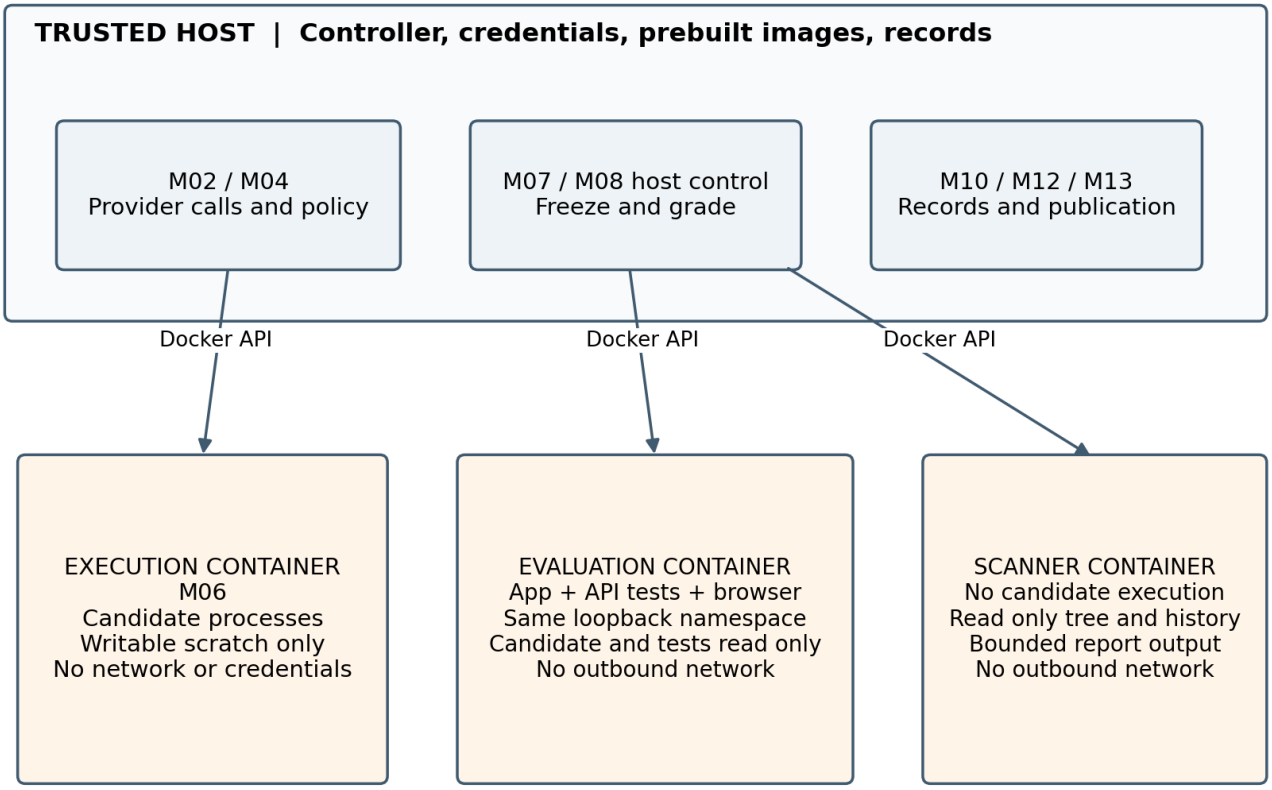


Figure 3. Target deployment. Candidate execution is untrusted; evaluator and scanner processes are isolated even though their definitions are team-owned.

The trusted host holds provider credentials and the Docker control interface. Neither is mounted into candidate or evaluator containers. The execution container has a non-root user, no outbound network, a writable disposable workspace and enforced CPU, memory and process limits.

For API/browser grading, the candidate server and evaluator client run inside the same disposable evaluation container and use its loopback address. Docker's none network driver supplies container-local loopback [7]; the host does not reach it through host loopback. Candidate and evaluator code are mounted read-only; temporary runtime state uses dedicated disposable storage. The agent loop stops before tests become available. Candidate and evaluator use separate non-root users, isolated writable directories and denied process tracing. M08 reads only the evaluator process result, never candidate stdout as a trusted report.

An independent scanner container reads the frozen tree and relevant history without executing candidate code. Container isolation reduces exposure but shares the host kernel; hostile-code workloads require an appropriately isolated operator machine or VM. This design does not claim containers are an absolute security boundary.

B uses the vendor's actual execution environment. A snapshot limits supplied input but does not enforce network or host filesystem isolation. Record approval settings, sandbox settings and observed deviations separately. It is not evidence that C isolation requirements passed.

8 Controlled execution workflow

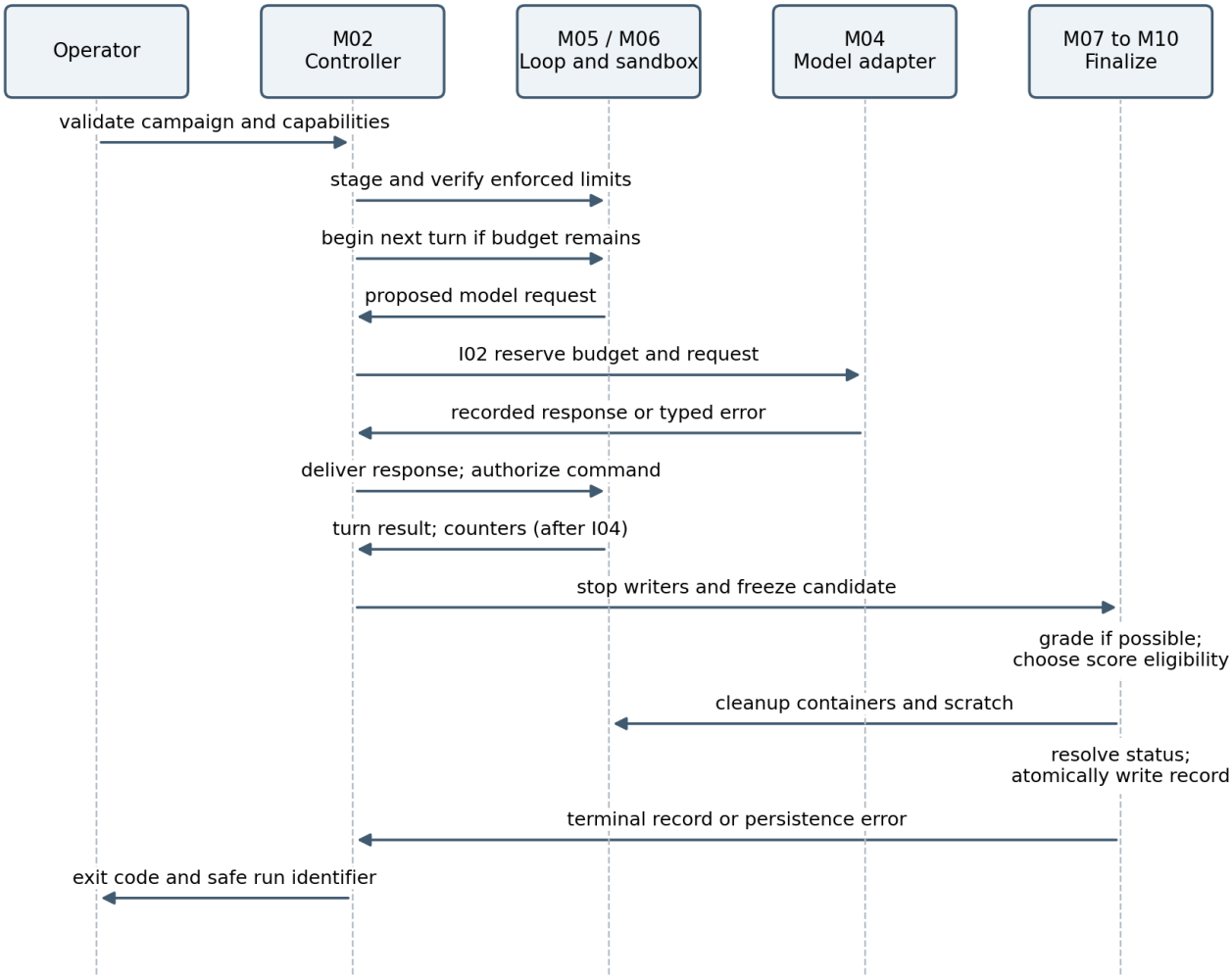


Figure 4. Approach C sequence. Budget checks apply before every provider request, retry and command; only the host performs provider calls.

An accepted run receives a unique ID and durable start journal after validation. The controller records monotonic elapsed intervals. Approval wait is excluded from active agent time; setup, grading and cleanup remain separately measurable. A turn is one agent response and its associated commands, consistent with the SRS definition.

Every exit path enters finalization. The controller stops agent activity, freezes any available candidate, grades when safe, tears down transient resources, resolves status, then writes the terminal record. An abrupt host crash is recovered from the start journal; no design can promise a completed write when durable storage itself is unavailable.

9 Native workflow and scenario evidence

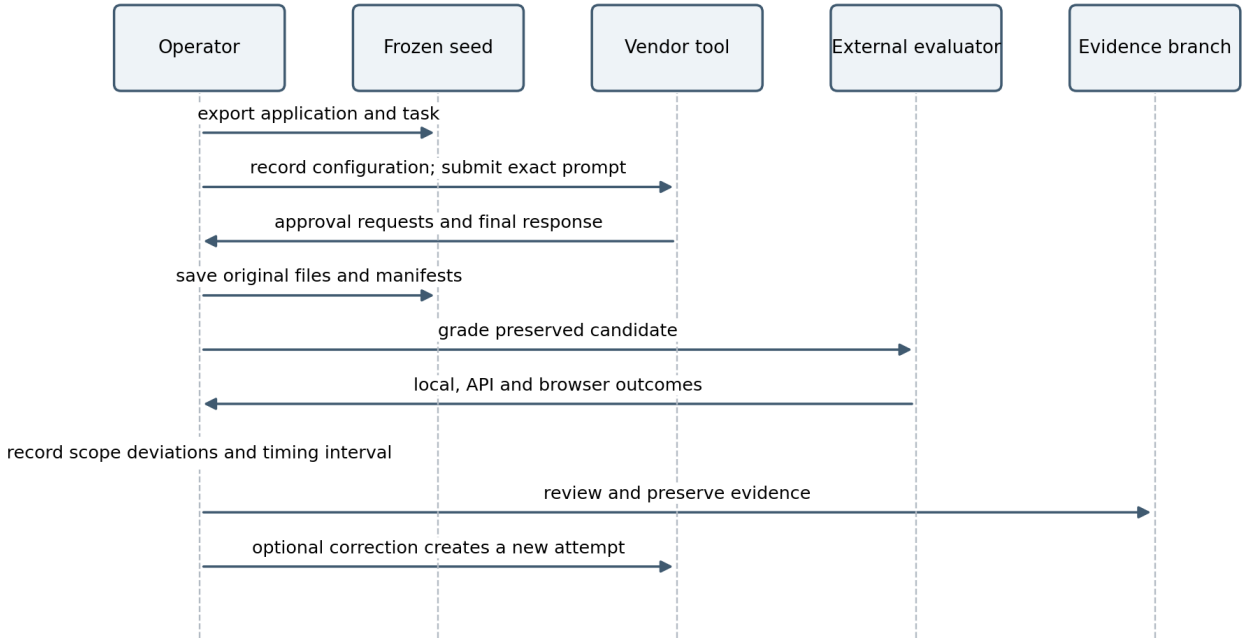


Figure 5. Approach B sequence. Snapshot preparation is a reproducibility step, not an enforced sandbox claim.

Task 001 starts from benchmark-start-v1; Task 003 starts from benchmark-start-v2-edit. Each operator records the actual tool/model settings and evaluator revision, submits the shared prompt, preserves original files before corrective feedback, and runs external acceptance suites. A corrected attempt has its own identity linked to the original.

DELETE scenario: the supplied evidence reports that the original Antigravity attempt passed the nine main tests but failed the later leading-zero check; one correction resolved it. Preserve both outcomes and identify the supplement's introduction. Do not describe the original attempt as passing every independent check.

Title-edit scenario: the supplied records report all four implementations passing 12 API and 10 browser tests and retaining the nine baseline tests. Local test totals differ because tools added different tests. These reported outcomes establish task correctness under recorded conditions, not security compliance, equal execution conditions or repeatability across trials [1,4].

Pilot times have different start/end definitions. Retain the original values and methods, but do not rank speed from them. Future runs record prompt submission to final response, approval wait, grading and setup separately.

10 Finalization and failure semantics

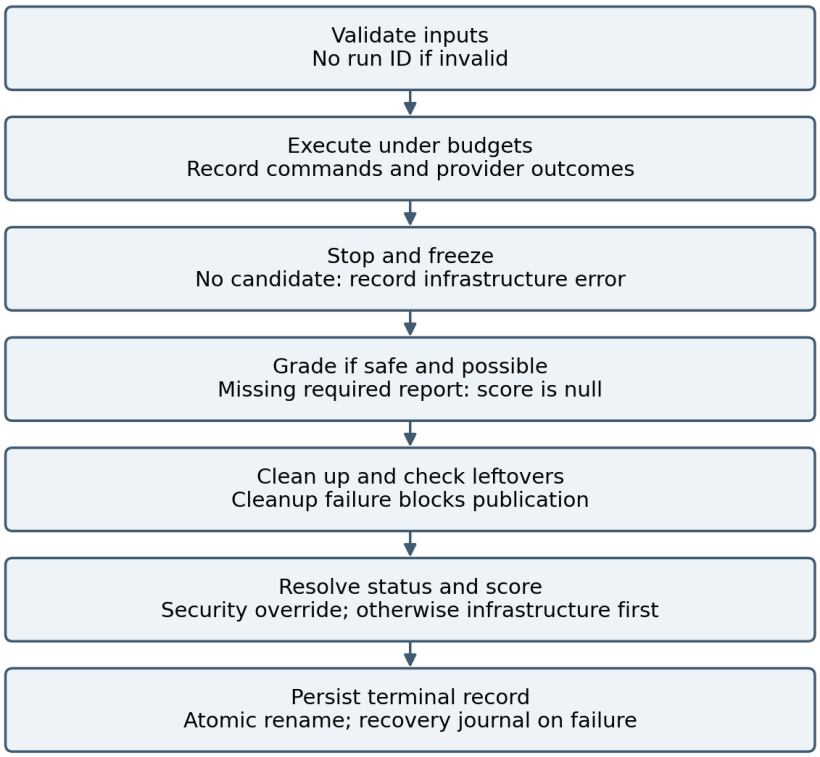


Figure 6. Target lifecycle activity. Status and scoring are resolved after report completeness and cleanup are known.

Condition in precedence order	Terminal treatment
Verified secret or forbidden action	failed; total 0; zeroed_by set. Missing scans still block publication.
Provider, environment, required grader or cleanup failure	infrastructure_error; scores null; one infrastructure rerun with new run ID.
Active budget or command watchdog exceeded	timeout; grade available candidate when safe; record distinct reason.
Approval rejected/expired or user cancellation	aborted; preserve available evidence.
Token cap reached or acceptance checks fail	failed; token_limit_exceeded or acceptance_failed reason.
All required acceptance and policy checks pass	passed; publication still requires license, redaction and review gates.

11 Data architecture

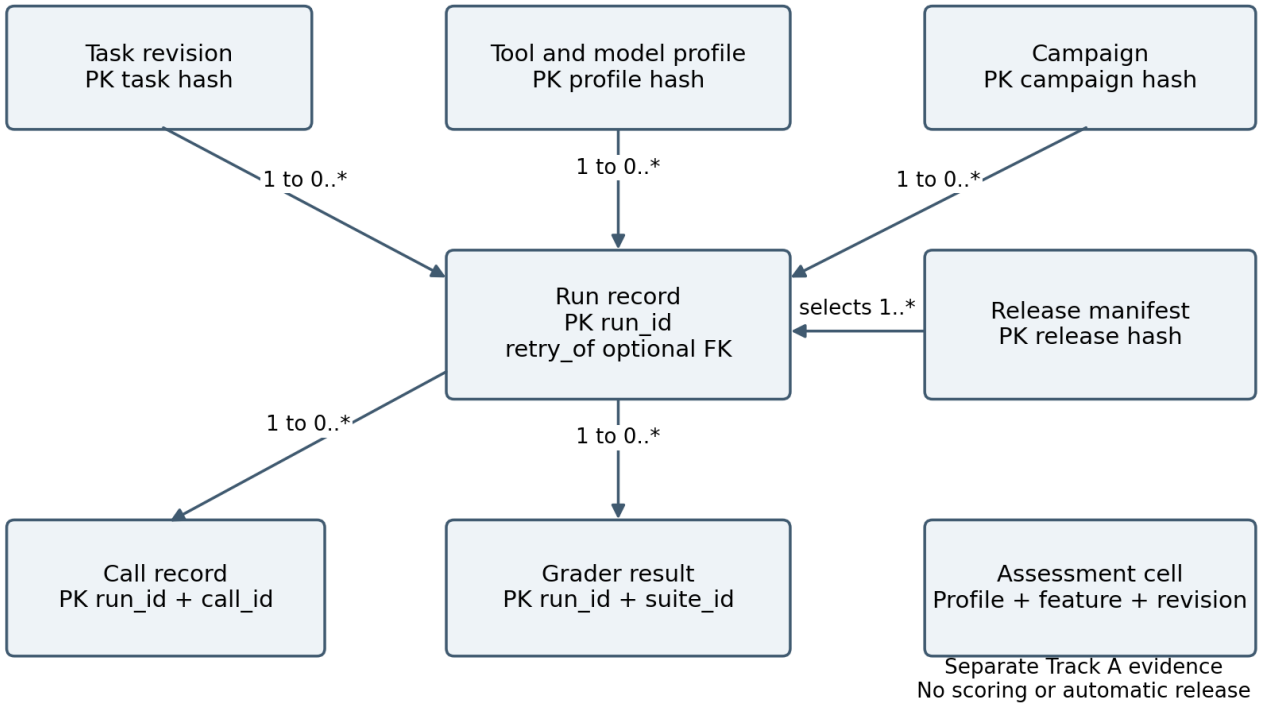


Figure 7. Conceptual data model. A run may have zero grader results when execution fails before grading. Release membership is separate from run existence.

Persistence uses versioned UTF-8 JSON and immutable evidence files. The trusted writer stages a file in the same filesystem as its destination, validates structure and semantics, flushes it, then atomically renames it. Readers see either the previous complete file or the new complete file. The recovery journal covers interrupted runs; it contains only sanitized metadata.

Candidate provenance includes the original bytes and a SHA-256 manifest. Evaluator identity includes a fixed Git commit plus raw file hashes. If LF-normalized hashes are used to compare text files across operating systems, store them separately with the normalization algorithm; never replace the original-byte hashes or normalize binary files.

Records are append-only by run ID. Corrections and infrastructure reruns reference an earlier run, never overwrite it. Git commits protect historical reviewability; cryptographic hashes identify content but do not prove the truth of the claims inside it.

Canonical target paths are benchmark-results/<approach>/<run_id>/ for empirical evidence and site/data/<release_id>/ for released data. Track A uses its own assessment records. Existing historical results branches are retained and linked; this path convention does not retroactively move or rewrite them.

12 Run record contract

The following is the proposed schema 3 design. It is an explicit amendment candidate to SRS Appendix A schema 2, which requires numeric usage and scores where unknown or infrastructure-error states need null. The team must approve and version the schema before claiming schema-3 conformance. All required fields below are always present; nullable fields carry an explanatory reason.

Field	Type and constraint	Meaning
schema_version / run_id	3 / unique string	Record contract and immutable identity.
approach / attempt / phase	B or C / integer ≥1 / original, correction, infrastructure_retry	Method and attempt identity; C campaign limits attempts to the approved plan.
retry_of	run ID or null	Prior attempt; cannot reference itself; same task lineage.
campaign_hash / task_hash / profile_hash	SHA-256 strings	Resolve to immutable input definitions.
baseline_commit / evaluator_commit / submission_commit	Git object IDs; submission may be null until committed	Reproduce inputs and evaluated candidate.
status / error	C: five SRS statuses; B: null / typed error or null	B suite outcomes do not impersonate a controlled run status.
turn_count	integer ≥0 or null	C max 5; B unknown requires reason.
timing	Timing object	Wall, active, approval, setup, grading and cleanup durations.
usage / cost	Objects with nullable numeric values	Unknown data are not zero; provenance and reason required.
human_involvement	Counts and event references	Approvals, rejected requests and corrective prompts separately.
candidate / grader_results / calls	Candidate object or null / arrays	Immutable hashes, reports and provider-call records.
scorecard	Score object or null	Null for B and infrastructure errors; policy version required otherwise.
publication	Eligibility object	Eligibility, blocking reasons, review reference and selected release.

All records reject unknown top-level fields, duplicate IDs, negative durations, invalid hash encodings and unresolved required references. JSON Schema handles structure; the host validator enforces reference integrity, totals, status/score invariants and permitted publication. An invalid record is quarantined rather than “repaired” into a success.

13 Supporting data contracts

Object	Required fields and invariants
TaskManifest	task_id, revision, baseline_commit, prompt_hash, allowed_paths[], baseline_test_hashes[], suite_ids[], dependency_lock_hash. Paths relative to the application root; reject traversal, external symlinks and unlisted edits.
Campaign	campaign_id, approach, ordered task/profile references, evaluator commit, image digests, policy/metric versions, repetitions, spend/rate caps, price reference and environment class. No scored C campaign starts with unresolved placeholders.
ToolProfile	tool, version or unknown_reason, interface, mode, model/version, reasoning setting, approval/sandbox policy, integrations, operating system and capability report. Distinguish observed configuration from documented availability.
Timing	wall_seconds, active_seconds, approval_wait_seconds, setup_seconds, grading_seconds, cleanup_seconds, measurement_method, interval_definition. Nullable unavailable fields require reason; avoid summing overlapping intervals.
Usage and cost	input_tokens, output_tokens, cached_tokens where reported, completeness, source, unknown_reason; estimated_usd, currency, price_version and billing interpretation. Record cached and uncached categories without double counting.
GraderResult	suite_id, suite_hash, candidate_hash, tool_version, outcome (completed/error), counts or null, findings[], duration_seconds, report_hash, error or null. Completed test counts sum consistently; scanner errors have no passing count.
Candidate	tree_hash, file_manifest[], allowed_change_result, baseline_test_preservation, capture_method, original_attempt_ref. Captured after writers stop; unavailable candidate is null with reason.
Error	code, stage, sanitized_message, retryable, evidence_ref. Stable codes; no raw prompt, credentials, host paths or stack dumps in public records.
Publication	eligible boolean, blocked_reasons[], review_ref or null, release_hash or null. A passing task alone never makes a record eligible.

Call records contain call_id, attempt number, start/end metadata, provider outcome and nullable usage. Failed calls are recorded; retries keep a parent request ID but have distinct call IDs. Usage totals are complete only when all chargeable calls are accounted for. Partial counts are retained as partial and do not support an asserted exact cost.

14 Interface contracts for execution

I01–I09 are target contracts. Internal transport is a typed in-process call; persisted forms are UTF-8 JSON. Expected failures return Result<T, Error>. Unexpected exceptions are caught at the controller boundary and converted to infrastructure_error after finalization. No interface permits an unvalidated path to become a host shell command.

Interface	Request and response	Preconditions and errors
I01 M01 → M02	ValidatedInputs{campaign, task, profiles, policy, suite_refs, hashes} → AdmissionResult{accepted, validation_errors[]}.	All immutable references resolve; no unresolved caps. INVALID_INPUT, HASH_MISMATCH or CAPABILITY_MISSING prevents run allocation.
I02 M02 → M04	ModelRequest{run_id, request_id, messages, output_token_cap, reservation_id} → ModelOutcome{call_id, response or error, usage, latency}.	M02 reserves tokens/cost and rate slot first. Messages are transient and excluded from telemetry. PROVIDER_ERROR retried at most twice; unknown usage blocks exact budget guarantees.
I03 M05 → M06	CommandRequest{run_id, command_id, script, cwd_relative, approval_ref, timeout_seconds} → execution handle.	M02 authorization and validated cwd required; no host execution. Max 60 s per command, also bounded by active deadline. FORBIDDEN_ACTION, APPROVAL_REQUIRED or SANDBOX_ERROR.
I04 M06 → M05	CommandOutcome{command_id, exit_code or null, sanitized_output, timed_out, duration, policy_events[]}	Output combined and bounded to 2,000 characters after redaction. Null exit code requires execution error. Watchdog and policy events are also delivered to M02.

The external provider exchange is HTTPS owned by M04, not I02's internal transport. Provider credentials are host-only. Failed request retries reuse the logical request identity but record separate attempts and actual usage. The adapter must not automatically retry after a hard budget stop.

Dangerous commands trigger the SRS escalation path. Operator approval does not override hard policy prohibitions such as outside-workspace writes, credential access, network egress or evaluator tampering. Commands remain untrusted data executed only inside M06.

15 Interface contracts for evaluation and release

Interface	Request and response	Preconditions and errors
I05 M07 → M08	FrozenCandidateRef{run_id, tree_hash, manifest_hash, suite_refs, private_storage_handle} → EvaluationBundle.	No active writers; candidate and grader hashes match. Handle is host-local and never public. FREEZE_ERROR or HASH_MISMATCH prevents grading.
I06 M08 → M09	EvaluationBundle{candidate_hash, grader_results[], required_suites[], policy_events[]} → ScoreEligibility.	Every required result must be complete and identify the same candidate. Missing/error reports → unscored. VERIFIED_SECRET or FORBIDDEN_ACTION records zero override and blocks publication.
I07 M09 → M10	ScoreDecision{status_candidate, dimensions or null, total or null, total_max or null, metric_version, reasons[]}	M02 supplies final cleanup and lifecycle outcomes before write. Range, weight and status invariants checked. M10 rejects SCHEMA_ERROR and inconsistent totals.
I08 M10 → M13 via M12 review	ReleaseInput{record_refs[], review_refs[], trusted_ci_refs[], comparison_group} → ReleaseOutcome{release_hash or errors[]}.	Record/schema/hash integrity, trusted CI, review, redaction and license gates pass. M11 is excluded pending a separate publication decision. On failure keep old release.
I09 tool adapter → M02	CapabilityReport{headless, model_pinning, sandbox_routing, turn_counting, usage_reporting, budget_control, evidence_refs} → AdmissionDecision.	Capability booleans describe tested integration, not marketing claims. Unknown/false required capability excludes adapter from C and keeps it eligible for separately labeled B.

M12 and M13 exchange files through repository/CI artifacts; network transfer uses authenticated GitHub operations. Reports are parsed against a size-bounded schema and tied to the exact candidate and workflow run. They are never evaluated as code. Release actions hold only the permission needed for their particular write.

Module-to-module data do not expose raw host paths. A private_storage_handle is an opaque controller-owned identifier resolved only inside trusted host code. Public references use repository-relative evidence locations and content hashes.

16 Track A data and assessment workflow

M11 implements SRS F6 and Specs 5.1–5.5 using the Feature Matrix Specification v0.2 [3]. The catalog contains MATRIX:F01–F18. The current specification describes two Copilot profiles, while working project materials leave the final profile set unresolved. The DDD therefore uses 18 × approved profile count and does not assert a completed 36-cell assessment.

Record	Fields and rules
Profile	Stable configuration ID; tool/harness; categories; modes; interface and execution location; model support; account/platform; included/excluded components; permissions; documentation coverage; access date and evaluator. Unknown source version stays unknown.
Assessment cell	Feature and profile IDs; one primary status; supported, unsupported and unestablished scope; setup; activation; evidence basis; control mechanism; source references or gap reason; limitations; assessment date; evaluator and different reviewer.
Evidence entry	Stable MATRIX:E identifier; publisher; page title; canonical URL; section; access date; source version/update date when available; short supported claim.
Review	Evaluator and different reviewer, reviewed cell revision, decision and material unresolved issues. No fabricated reviewer or approval date.

Allowed statuses are Built in; Configuration or extension required; Custom implementation required; Unavailable; and Not established from documentation. They are delivery categories, not ordered quality scores. Missing documentation does not imply absent capability.

Workflow: define profiles and exclusions; initialize all catalog cells; inspect official sources; record claim-specific support or uncertainty; obtain independent review; deliver the sourced assessment. No task execution, screenshots from team tests or benchmark outcomes are used to validate a matrix claim.

Completion requires a status and rationale for every defined cell, source references for support/absence claims and a recorded reviewer. Uncertain cells can be complete when the gap is documented. Any presentation states “documentation review,” access dates and separation from empirical results. Automated microsite integration is an open design extension, not a prerequisite for Track A completion.

17 Testbed APIs and demonstration behavior

The reference application uses Express and an in-memory todo array. Restarting the server resets the baseline. Reload persistence means the edited title survives a browser refresh while the same server process runs; it does not mean persistence across server restart. Task-specific frozen seeds define which features are initially absent.

Endpoint or behavior	Contract and validation
GET /api/todos	200 with JSON array of {id:number, title:string, completed:boolean}. Each isolated test starts from the recorded baseline state.
POST /api/todos	201 with new todo; reject missing/blank title with 400. Preserve baseline behavior and task-specific validation.
PUT /api/todos/:id	Update permitted fields; 200 for existing todo, 404 for nonexistent valid ID. Title-edit task adds validated title update while preserving completion state; exact malformed-input rules come from pinned task/evaluator.
DELETE /api/todos/:id	Task 001 adds strict positive safe-integer ID handling; digits such as 01 are valid. Invalid 0, negative, decimal or mixed characters → 400; existing → 204 empty body; missing/repeated → 404.
Title editing UI	Enter edit mode; save valid title; display validation/server failure; cancel without mutation; preserve completed state and existing Add/toggle/Delete interactions.
Rendering and accessibility	Render user titles as text; support keyboard operation, accessible control names and visible error feedback. Browser checks validate task behavior, not full WCAG conformance.

The evaluator starts the application on loopback with a fresh process and a controlled port. API and browser tests live outside the candidate workspace during generation. After freeze, prepared evaluator files are read-only. Test dependencies and browser builds are pinned; a substituted browser build is a disclosed environment deviation.

For Chad’s demo, restore one preserved submission onto its matching frozen seed, install from its lockfile in the prepared demo environment, run the independent checks, then demonstrate editing, validation, cancellation and refresh persistence. Show the common CI comparison only after it has actually completed. The professor’s presentation must explain the architecture and trade-offs as well as show behavior.

18 Development and publication views

Repository area	Design responsibility
app/ and baseline/	Testbed and historical baseline copies; campaign uses explicit frozen task commit.
pipeline/autogen/harness/	Prototype contracts, orchestration, staging, provider wrapper, execution, telemetry, scoring and export. Target boundaries map to M01–M10 and M12.
pipeline/autogen/graders/ and tests/	Evaluators and harness verification. Candidate tests never replace external acceptance suites.
pipeline/tasks/	Task prompts/manifests; task revisions are immutable campaign inputs.
course_artifacts/sources/	Canonical course Markdown; DDD, SRS, matrix specification and assessment have distinct ownership.
.github/workflows/	CI grading and deployment definitions; trusted revision pinned per campaign.
benchmark-results/ and site/data/	Proposed immutable empirical records and release outputs; not a claim that the current site already reads them.

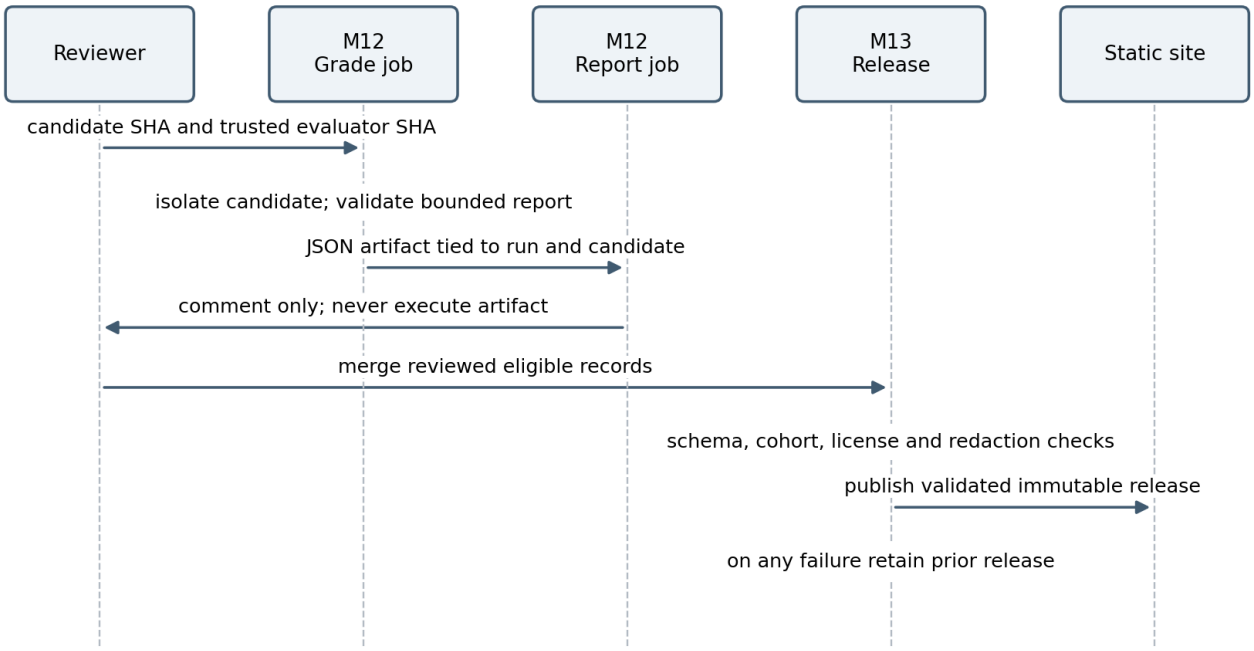


Figure 8. Proposed review and publication sequence. The report job never runs candidate code or artifact contents.

Build dependencies are lockfile-pinned. Prepare and cache evaluation images before the offline run; use npm ci rather than unbounded installation during grading. Grade jobs execute candidate code without secrets or write tokens. Separate reporting and release jobs validate their input artifacts; privileged jobs never check out and execute untrusted candidate code [8].

19 Scoring and comparison interpretation

Weighted scoring applies only to C after harness verification and metric-contract approval. B reports raw suite outcomes and human involvement. A has no numerical score. The formulas below preserve the SRS's proposed policy; they are not measured scores for the pilot runs.

Dimension	SRS formula or rule
Functionality F, 35%	100 × passed external acceptance assertions / total assertions. Suite inventory fixed before run.
Security S, 25%	max(0, 100 − 25 × new high/critical findings relative to baseline). A secret or forbidden action overrides the total to zero.
Quality Q, 20%	Four 25-point checks: no new formatter violations; complexity ≤10 unless already above baseline; no additional duplicated lines; test runtime ≤1.2 × baseline. Tools and measurement method fixed in metric contract.
Tokens T, 10%	100 when U ≤0.5L; otherwise max(0, 200 × (1 − U/L)), with U used tokens and L approved limit.
Human help H, 10%	max(0, 100 − 25 × avoidable interventions). A second reviewer applies the written rule; routine approval is recorded separately.

Total = 0.35F + 0.25S + 0.20Q + 0.10T + 0.10H. If any included configuration lacks complete token counts, remove T for every configuration in that comparison group: totals remain out of 90, without renormalization. Infrastructure errors have null scores, not zero. The schema amendment must support these values.

Average the planned original attempts per task, then average task means with equal task weight. Include agent failures; keep corrections as separate assisted evidence. Infrastructure failures receive one linked rerun and unresolved failures are excluded with the resulting sample count disclosed. Report min–max range and pass/fail instability; three runs do not establish statistical certainty.

Cost is an estimate from attributable usage and the pinned price schedule. Cached, uncached and output billing categories must match the provider's reporting. Missing usage, subscription access or incomplete billing information means cost is unknown or qualified; no token count is presented as an exact energy measurement.

Native pilots are an initial sample on one application. Operator knowledge, unequal approvals, different models, differing browser builds and inconsistent timing intervals limit causal interpretation. A common CI regrade improves evaluation consistency; it does not recreate the original generation conditions.

20 Performance capacity and timing

Target or estimate	Basis	Validation approach
2 CPUs, 2 GB RAM, 128 processes; non-root	SRS Spec 1.3 target	Host-observed resource tests and configuration inspection.
Command watchdog 60–61 seconds	SRS Spec 1.3 target	Run controlled overlong command and inspect monotonic deadline/termination.
Ready <3 s; teardown <5 s	SRS Spec 1.4 target; cached image assumption	Measure lifecycle after normal exit, cancellation and injected error.
≤5 turns; ≤120 s active	SRS Spec 2.1 target for C	Count turns; inject deadline crossing; verify no further calls.
Approval expires 180 ±1 s	SRS Spec 2.4 target	Measure interactive/headless unanswered approval and excluded wait time.
Native tests <500 ms; UI ≤100 ms after response	SRS Specs 3.1–3.2 targets	Repeated test timing and browser response-to-DOM measurements on declared host.
Telemetry overhead <5%	SRS Spec 4.2 target	Ten matched runs with telemetry on/off; disclose variance.
CI grading ≤120 s; site update ≤2 min	SRS Specs 4.5 and 4.4 targets	Measure start/end events, separating queue delays and documenting target interpretation.
96 runs ×120 s =3.2 h active maximum	SRS campaign planning estimate	Excludes setup, waits, retries, grading and cleanup; not total wall time.
5 MiB/run ×96 =480 MiB	Unmeasured storage planning estimate	Measure record/evidence sizes; separate text records from logs, videos and images.

One run executes at a time. Host planning assumptions are 16 GB RAM and 100 GB free disk from the supplied DDD; these are not measured minimum requirements. Campaign wall time equals setup + active execution + approval waits + grading + cleanup + retries and scheduling overhead. The earlier 3.5-hour estimate is removed because grading duration was unmeasured.

Scaling first adds sequential campaigns on a compatible larger host. Parallelism requires replacing global state, reserving aggregate budgets atomically, isolating ports and resources, and measuring contention. It would create a new environment class rather than silently mixing measurements.

21 Security observability and recovery

Security and integrity

M03 uses manifest-only export, rejects credentials and links escaping the root, and supplies no host environment. M06 enforces resource and network restrictions. Network isolation alone does not prove that every denied attempt was recorded: the target also requires host-observed connection-attempt instrumentation validated with TCP, UDP and ICMP probes. If that observation cannot be implemented reliably, C admission remains blocked for Spec 1.1.2.

M07 compares every candidate path and preserves baseline test content. M08 scans the frozen tree and available history, including secrets deleted in later commits. Because generation snapshots omit Git metadata, the host records input history and the generated patch lineage; limitations on intermediate unrecorded edits must be disclosed. Missing scanner, incomplete history coverage or malformed report blocks the claimed audit result.

Observability

Structured events contain run_id, event_id, stage, monotonic duration, safe error code and evidence hash. Counts cover turns, provider retries, approval waits, grader outcomes and cleanup attempts. Records exclude prompts, environment values, credentials, personal information and internal paths. Redaction occurs before serialization, with canary validation and independent review before public release.

Per-run stage events provide tracing without requiring an always-on telemetry service. Local summaries expose incomplete runs, invalid records and budget state. Raw vendor transcripts are separate pilot evidence with explicit review; existing transcript publication does not establish SRS telemetry compliance.

Fault tolerance and configuration

Provider calls retry at most twice with bounded backoff subject to remaining budgets. Controller finalization runs after handled errors and cancellation. A recovery sweep reconciles start journals without terminal records, removes named disposable resources and writes an infrastructure-error record when storage is available. Persistence failure returns a nonzero operational exit and blocks release.

Effective configuration is resolved and validated once, hashed, and attached to the campaign. Secrets are injected only into the host provider adapter. Unknown usage and price information prevents a hard cost guarantee; admission requires a conservative bound or an approved alternative policy. No implicit defaults fill missing approved token, spend or rate caps.

22 Architecture decisions for execution

These dispositions describe the target design. A proposed disposition is not a recorded team approval. Each ADR preserves its original identifier; the attachment's native-pilot ADR-13 is separated from the repository's Track A ADR-13 as explained below.

ADR 01 Separate comparison approaches

Proposed. A combined leaderboard would mix documented capabilities, uncontrolled native pilots and controlled scores. Separate datasets preserve interpretation at the cost of a less compact summary. SRS F1–F6 and scenario evidence justify this separation.

ADR 02 Local modular application

Proposed. A local modular monolith fits the sequential 96-run plan and avoids network services, queues and credentials. Microservices improve independent scaling but add deployment and failure modes without a demonstrated workload need. A single host remains a shared failure domain; disposable execution and recovery journals mitigate it.

ADR 03 Enforce limits on the host

Target; partial prototype behavior reported. Prompt instructions cannot reliably enforce budgets or isolation. Host counters, watchdogs and container quotas cost integration work but make violations observable and testable. Admission fails when required enforcement is unavailable.

ADR 04 Use evaluator owned suites

Used in reported pilots; protected-container design proposed. Candidate-authored tests are useful regression evidence but cannot define functionality scores. Fixed external tests prevent self-grading; independent review and adversarial fixtures address defects in the evaluator itself.

ADR 05 Isolate the orchestration framework

Proposed boundary. Keep framework-specific code behind M05/M06 interfaces rather than migrate during document preparation or write an entire new loop. This reduces migration cost but adds adapter maintenance. Evaluate a replacement only against required capabilities and regression evidence; no unsupported lifecycle claim is needed.

ADR 06 Version the record schema

Proposed SRS amendment. Schema 3 represents unknown usage and unscored infrastructure failure explicitly. Keeping schema 2 would require fabricated zeros or lost information. The cost is migration and dual readers. Historical records retain their original version and conversion provenance.

23 Architecture decisions for evidence and release

ADR 07 Version the metric contract

Proposed. A versioned scoring policy prevents formula drift and incomparable averages. It requires recomputation labels and immutable policy references. The DDD retains SRS weights and unknown-token treatment; no pilot is retroactively assigned a weighted score.

ADR 08 Wrap the provider client

Reported prototype choice; target retained. An in-process adapter can attribute latency and usage to C calls without a separate proxy. A proxy offers broader routing but adds configuration and a failure point. The wrapper does not measure native vendor sessions; B retains tool-reported usage and limitations.

ADR 09 JSON records and static reporting

Proposed. JSON supports reviewable diffs and immutable evidence without database operations. A relational store would improve queries and concurrent writes but introduces service or binary-diff costs. Cross-file integrity and aggregation move to the validator/build step. A generated database can be considered if measured volume justifies it.

ADR 10 Export patches onto trusted history

Proposed. Apply an allowlisted patch to the recorded upstream revision rather than pushing arbitrary scratch history. This costs conflict handling but preserves provenance and prevents unintended configuration/history changes. Reject conflicts; do not silently resolve them into a different candidate.

ADR 11 Separate grading and publication privileges

Proposed. Candidate execution has no write token; reporting and publishing use separate minimal permissions. This adds artifact validation and execution time but prevents untrusted code from inheriting publication authority. Pin reviewed dependencies; verify every report’s workflow and candidate identity [8].

ADR 12 Preserve original and corrected attempts

Used in reported pilots. Separate attempts retain correction count and exact intervention text in reviewed evidence. Overwriting the original hides human contribution. Corrected results may be shown beside originals but do not replace the original trial in reliability summaries.

24 Track A decision and requirements reconciliation

ADR 13 Preserve documentation only assessment

Proposed target aligned with the repository DDD and SRS F6. M11 uses official documentation and independent review; unknowns remain explicitly unknown, with no weights or performance ranking. Live testing would answer a different question and is excluded. The profile set must be resolved against the matrix specification before cells are populated.

ADR 14 Preserve native pilots as a separate workflow

Proposed identifier for the native-tool decision called ADR-13 in the attached PDF. This renumbering resolves its collision with the repository's existing Track A ADR-13. Native pilots share task/evaluator inputs while retaining tool-specific models and approvals. Immediate full integration would delay evidence and exclude unsupported tools; dropping C would abandon controlled requirements. The design retains both with separate claims and admission checks.

Decision	Required disposition before final claim
SRS V2 baseline	Identify approved V2 revision/commit; compare every traceability row and obtain team approval of changed obligations.
D05 schema contract	Approve schema 3 or amend schema 2 to represent unknown usage, null scores and B evidence. Until then no schema-3 compliance claim.
D06 baseline selection	Use task-specific frozen commits for pilots. Approve a clean canonical C seed and baseline suite; do not merge historical outcomes to disguise seed differences.
D09 retention	Approve retention, access and deletion rules for raw evidence. Public records remain redacted; transient execution data are cleaned after controlled finalization.
Budget and rate policy	Resolve $100 versus SRS credit wording; fill token/rate/spend placeholders and price source. No scored campaign starts with undefined caps.
Active time	Retain SRS 120 seconds for C until amended. Longer B elapsed times do not by themselves justify changing an active-time requirement.
Matrix profiles and publication	Approve profile set; preserve 18-feature catalog. Decide automatic M11-to-M13 integration separately.
NFR references	Resolve missing SRS NFR definitions and TC-4.5-3 reference. Existing concrete spec clauses remain mapped; undefined references are not treated as verified.

D05, D06 and D09 are defined here as unresolved decisions, replacing dangling labels. Historical V02–V05/E15 assertions are not carried as verified current findings; source [1] retains them for provenance. Recheck implementation against a specific commit before asserting them in a final progress claim.

25 Public health safety and wider impacts

The system supports decisions about software-generation workflows. Its direct public-safety contribution is controlled evaluation and accurate reporting, not certification of generated software for deployment. The following design responses are targets; native pilots establish only the checks actually recorded in their evidence.

Factor	Design response and validation
Public health and safety	Never deploy candidate code to production. Isolate execution, deny credentials/network access, limit resources and independently grade. Verify controls with SRS sandbox and auditing tests before controlled evaluation.
Public welfare	Publish task scope, configuration, interventions and limitations. Avoid presenting a small sample as universal production readiness. Review charts and summaries for unsupported winner claims.
Global	Record region/account/platform and model availability where material. Do not assume tool access or pricing is uniform. Portability is tested per supported platform; English-only tasks limit generalization.
Cultural	Acknowledge variation in engineering practice and AI supervision. Document prompts and review criteria; disclose AI-assisted authorship and retain human responsibility for judgments.
Social	Minimize personal and sponsor data; use accessible controls and readable evidence. Separate documented capability from measured effectiveness to support fair interpretation.
Environmental	Bound runs and retries, use existing workstations and static publication. Record usage as a workload indicator, not measured energy or emissions. Avoid unnecessary reruns while preserving reproducibility.
Economic	Validate spend and rate caps before execution; disclose subscription/API differences and uncertain cost. Reuse supported open-source components; scan license provenance and block flagged code publication.

The prototype is a semester-scale research and engineering tool on one JavaScript application. Results do not establish suitability for medical, transportation, infrastructure or other safety-critical software. Security and license scanners can miss issues; human review and clearly stated scope remain necessary.

SRS V2 should record these stakeholder needs and constraints explicitly. This DDD maps their implementation responses without inventing approval of a revised legal or institutional policy. The design supports review of applicable obligations; it does not claim legal certification.

26 Risk register and validation strategy

Risk	Mitigation and validation	Residual limitation
Model and operator confounding	Separate B/C; record profiles and interventions; use matched conditions and repeated trials where supported.	Native workflows cannot isolate the harness contribution.
Candidate escapes intended boundary	Manifest staging, host-only credentials, offline non-root containers, quotas and isolated host/VM. Validate Spec 1.1–1.4 probes.	Shared-kernel risk; snapshot alone is insufficient.
Evaluator tampering or weak tests	Freeze candidate; trusted suite commit; readonly mount; adversarial fixtures; exact suite inventory.	Coverage is bounded to authored cases.
Provider outage or unknown usage	Bounded retries, typed error, explicit nulls, budget admission and infrastructure rerun.	Provider billing visibility may remain incomplete.
Missing offline dependencies	Build/verify images from lockfiles before execution; pin browser/image digests.	First preparation cost excluded from cached lifecycle target.
Invalid or leaked evidence	Schema + semantic checks, redaction canaries, license/secret scan, independent release review.	Detection tools are imperfect; fail closed on missing reports.
Crash or cleanup failure	Start journal, finalizer, orphan sweep, idempotent cleanup and atomic record write. Inject crash/storage failures.	Hard storage loss can prevent evidence recovery.
Schedule and scope growth	Prioritize correct evidence path and testbed; defer distributed execution and optional matrix-site integration.	Incomplete C functionality remains visible.
Unstable performance results	Repeat tasks with fixed conditions; report sample size/range and timing method.	Small N does not support broad statistical claims.

Validation proceeds through component fixtures, adapter contract checks, boundary integration tests and end-to-end campaigns. Use the SRS harness checks H-0–H-7 before scored C evaluation; H-8 independently checks Track A by document inspection. The detailed test plan remains in the SRS rather than being duplicated here.

Architecture validation adds targeted cases for null usage, no grader results, cleanup failure, stale hashes, conflicting records, corrupted CI artifacts and release rollback. These are proposed design checks, not new approved SRS test IDs or completed test results.

27 Verification and release gates

Gate	Required evidence	What a failure means
Requirements baseline	Approved SRS V2 plus reconciled decisions and quantitative values.	Design remains reviewable, but final V2 conformance cannot be asserted.
Adapter admission H-0	Actual supported model/control route, turn counting, usage and isolation fixture results.	Tool remains B; do not relabel a generic model run as that vendor harness.
Harness verification H-1–H-7	Recorded results for baseline, boundaries, orchestration, independent grading, telemetry and reporting.	No scored C campaign or claim of validated controls.
Track A H-8	Complete profile/cell/source/reviewer records, correct uncertainty statements.	Assessment incomplete; no live testing required to close it.
Candidate correctness	Preserved candidate and fixed evaluator results; baseline regression preservation.	Failed task remains valid evidence but cannot merge as an accepted implementation.
Publication safety	Completed secret/license scans, valid redaction, review, trustworthy CI and matching hashes.	Keep record private/quarantined and retain prior public release.
Course submission	DDD, actual in-class deck, current timesheet and accurate AI disclosure; team review.	Document completeness alone cannot satisfy the full 90% presentation component.

A correctness CI workflow for four preserved title-edit implementations is a valuable next step. It verifies their code under a common evaluation environment. It does not replay the generation session, validate every sandbox requirement, establish a speed ranking or certify the broader harness.

The professor’s presentation should cover the same architecture, interfaces, trade-offs, traceability, operational numbers and public-impact factors. The sponsor demonstration emphasizes reproducible task behavior, evidence and the decision about the next comparison method. They can reuse figures but have different purposes.

Exit and operational reporting

Target CLI exit codes are 0 for passed, 1 for an agent/task failure or timeout, 2 for invalid input or infrastructure/persistence failure, and 130 for user cancellation. Approval expiry is an aborted run reported with nonzero exit. Preserve the record’s detailed status and reason rather than inferring everything from a process code.

Appendix A1 Requirements traceability

Design coverage of the repository SRS working baseline [2]. Clause IDs are listed explicitly; test IDs are the existing SRS cases for the specification, not claims that each clause has a unique or passing test. ADR references use the ADR number. All runtime validation remains pending unless separately supported by a run record.

Spec 1.1 Network Egress Boundary

Clauses: 1.1.1, 1.1.2. Modules: M06. Interfaces: I03 I04. Design: Host policy. ADR: 03.

Existing verification: TC-1.1-1, TC-1.1-2, TC-1.1-3, TC-1.1-4.

Design realization: M06 denies egress; host instrumentation records attempted connections and sends forbidden-action events to M02. Admission checks both enforcement and detection.

Spec 1.2 Secret Isolation and Zero-Token Boundary

Clauses: 1.2.1, 1.2.2. Modules: M03 M04 M06. Interfaces: I01 I02 I03. Design: Manifest staging. ADR: 03.

Existing verification: TC-1.2-1, TC-1.2-2, TC-1.2-3.

Design realization: M03 removes credential-bearing inputs; M04 performs model calls on the host. No host credential environment or profile mount enters M06.

Spec 1.3 Resource Quotas and Denial-of-Service (DoS) Defense

Clauses: 1.3.1, 1.3.2, 1.3.3, 1.3.4. Modules: M06. Interfaces: I03 I04. Design: Watchdog. ADR: 03.

Existing verification: TC-1.3-1, TC-1.3-2, TC-1.3-3, TC-1.3-4, TC-1.3-5.

Design realization: M06 applies CPU, RAM, process and non-root controls per container. Its watchdog kills overlong commands and reports the timeout reason.

Spec 1.4 Ephemeral Sandbox Lifecycle

Clauses: 1.4.1, 1.4.2, 1.4.3. Modules: M02 M03 M06 M10. Interfaces: I01 I03 I07. Design: Finalizer. ADR: 02 03.

Existing verification: TC-1.4-1, TC-1.4-2, TC-1.4-3.

Design realization: M03 uses a cached prepared image. M02 finalization and M06 cleanup cover normal completion, interruption and error; leftovers block release.

Spec 1.5 Fail-Closed Auditing

Clauses: 1.5.1, 1.5.2, 1.5.3. Modules: M08 M09 M10. Interfaces: I05 I06 I07. Design: Fail closed. ADR: 04.

Existing verification: TC-1.5-1, TC-1.5-2, TC-1.5-3.

Design realization: M08 audits frozen tree and relevant history. M09 records verified-secret zero overrides; incomplete scans remain unscored and unpublished.

Traceability limitation: missing SRS NFR definitions and the undefined TC-4.5-3 reference require requirements reconciliation. Spec 4.5.3 is still covered by the timing assertion in TC-4.5-1; this does not create a missing test ID.

Appendix A2 Requirements traceability

Design coverage of the repository SRS working baseline [2]. Clause IDs are listed explicitly; test IDs are the existing SRS cases for the specification, not claims that each clause has a unique or passing test. ADR references use the ADR number. All runtime validation remains pending unless separately supported by a run record.

Spec 2.1 Multi-Agent Turn Management

Clauses: 2.1.1, 2.1.2. Modules: M02 M05. Interfaces: I01 I02 I03. Design: State machine. ADR: 03.

Existing verification: TC-2.1-1, TC-2.1-2, TC-2.1-3.

Design realization: M02 owns counters and the active deadline. M05 cannot request another model turn after the limit; finalization still preserves evidence.

Spec 2.2 Execution Feedback Loop

Clauses: 2.2.1, 2.2.2. Modules: M05 M06. Interfaces: I03 I04. Design: Feedback loop. ADR: 05.

Existing verification: TC-2.2-1, TC-2.2-2, TC-2.2-3.

Design realization: M06 returns exit status and bounded sanitized output via I04. M05 delivers this feedback to the next permitted model turn.

Spec 2.3 Targeted File Patching

Clauses: 2.3.1. Modules: M01 M03 M07. Interfaces: I01 I05. Design: Immutable snapshot. ADR: 04 10.

Existing verification: TC-2.3-1, TC-2.3-2.

Design realization: M01 validates allowed paths and M07 compares the complete frozen manifest against the seed. Outside-scope changes create explicit policy events.

Spec 2.4 Human-in-the-Loop Escalation

Clauses: 2.4.1, 2.4.2, 2.4.3. Modules: M02 M05. Interfaces: I03 I04. Design: Approval state. ADR: 03.

Existing verification: TC-2.4-1, TC-2.4-2, TC-2.4-3, TC-2.4-4.

Design realization: M05 raises escalation after consecutive failed turns or dangerous commands. M02 separates approval waiting from active time and expires unanswered requests.

Spec 2.5 Cost and Rate Guardrails

Clauses: 2.5.1, 2.5.2, 2.5.3, 2.5.4. Modules: M02 M04. Interfaces: I02 I09. Design: Reservation breaker. ADR: 03 08.

Existing verification: TC-2.5-1, TC-2.5-2, TC-2.5-3, TC-2.5-4.

Design realization: M02 reserves budgets and rate slots before calls; M04 records every attempt and bounded retry. Undefined approved limits prevent admission.

Traceability limitation: missing SRS NFR definitions and the undefined TC-4.5-3 reference require requirements reconciliation. Spec 4.5.3 is still covered by the timing assertion in TC-4.5-1; this does not create a missing test ID.

Appendix A3 Requirements traceability

Design coverage of the repository SRS working baseline [2]. Clause IDs are listed explicitly; test IDs are the existing SRS cases for the specification, not claims that each clause has a unique or passing test. ADR references use the ADR number. All runtime validation remains pending unless separately supported by a run record.

Spec 3.1 Target API & State Determinism

Clauses: 3.1.1, 3.1.2, 3.1.3. Modules: Testbed M03 M08. Interfaces: I01 I05. Design: Fresh process. ADR: 02.

Existing verification: TC-3.1-1, TC-3.1-2, TC-3.1-3, TC-3.1-4, TC-3.1-5.

Design realization: M03 starts the pinned in-memory testbed in a fresh process. M08 verifies CRUD response contracts, reset determinism and native test timing.

Spec 3.2 Client UI Synchronization

Clauses: 3.2.1, 3.2.2. Modules: Testbed M08. Interfaces: I05 I06. Design: Client server. ADR: 04.

Existing verification: TC-3.2-1, TC-3.2-2.

Design realization: The browser client updates from server responses without navigation. M08 browser checks measure DOM update timing and required browser coverage.

Spec 3.3 Automated Regression Suite

Clauses: 3.3.1, 3.3.2. Modules: Testbed M03 M08. Interfaces: I01 I05 I06. Design: Frozen baseline. ADR: 04.

Existing verification: TC-3.3-1, TC-3.3-2, TC-3.3-3.

Design realization: M03 verifies the clean seed before generation. M08 retains baseline regression inventory and checks that deliberately broken fixtures are detected.

Traceability limitation: missing SRS NFR definitions and the undefined TC-4.5-3 reference require requirements reconciliation. Spec 4.5.3 is still covered by the timing assertion in TC-4.5-1; this does not create a missing test ID.

Appendix A4 Requirements traceability

Design coverage of the repository SRS working baseline [2]. Clause IDs are listed explicitly; test IDs are the existing SRS cases for the specification, not claims that each clause has a unique or passing test. ADR references use the ADR number. All runtime validation remains pending unless separately supported by a run record.

Spec 4.1 Evaluator-Owned Acceptance Suites

Clauses: 4.1.1, 4.1.2. Modules: M07 M08. Interfaces: I05 I06. Design: External oracle. ADR: 04.

Existing verification: TC-4.1-1, TC-4.1-2, TC-4.1-3.

Design realization: M07 freezes before M08 supplies protected external tests. No candidate-authored test can replace the acceptance oracle or alter its inventory.

Spec 4.2 Run Telemetry Validation

Clauses: 4.2.1, 4.2.2, 4.2.3, 4.2.4. Modules: M04 M10. Interfaces: I02 I07. Design: Atomic record. ADR: 06 08.

Existing verification: TC-4.2-1, TC-4.2-2, TC-4.2-3, TC-4.2-4, TC-4.2-5.

Design realization: M04 attributes calls and usage; M10 validates structure, totals, redaction and atomic persistence. Unknown usage requires the proposed schema amendment.

Spec 4.3 License Scan of Generated Code

Clauses: 4.3.1, 4.3.2. Modules: M08 M13. Interfaces: I06 I08. Design: Publication gate. ADR: 04 11.

Existing verification: TC-4.3-1, TC-4.3-2, TC-4.3-3.

Design realization: M08 records license findings; M13 excludes flagged candidates. Missing or failed scanning prevents publication rather than implying a clean result.

Spec 4.4 Automated Microsite Publishing

Clauses: 4.4.1, 4.4.2, 4.4.3, 4.4.4. Modules: M10 M13. Interfaces: I07 I08. Design: Static release. ADR: 09.

Existing verification: TC-4.4-1, TC-4.4-2, TC-4.4-3, TC-4.4-4.

Design realization: M13 reads validated committed records, computes compatible aggregates, includes AI/model metadata and promotes an immutable static release.

Spec 4.5 Trusted-Base Grading Workflow

Clauses: 4.5.1, 4.5.2, 4.5.3. Modules: M12. Interfaces: I05 I06 I08. Design: Privilege separation. ADR: 10 11.

Existing verification: TC-4.5-1, TC-4.5-2.

Design realization: M12 pins trusted grader code, rejects candidate grader changes and emits a required result for merge protection. Grading duration is recorded separately.

Traceability limitation: missing SRS NFR definitions and the undefined TC-4.5-3 reference require requirements reconciliation. Spec 4.5.3 is still covered by the timing assertion in TC-4.5-1; this does not create a missing test ID.

Appendix A5 Requirements traceability

Design coverage of the repository SRS working baseline [2]. Clause IDs are listed explicitly; test IDs are the existing SRS cases for the specification, not claims that each clause has a unique or passing test. ADR references use the ADR number. All runtime validation remains pending unless separately supported by a run record.

Spec 5.1 Profile Definition

Clauses: 5.1.1, 5.1.2, 5.1.3. Modules: M11. Interfaces: M11 records. Design: Profile catalog. ADR: 13.

Existing verification: TC-5.1-1, TC-5.1-2.

Design realization: M11 stores explicit documented profiles and exclusions. A missing published version is represented by source access date and uncertainty.

Spec 5.2 Feature Catalog and Status Labels

Clauses: 5.2.1, 5.2.2, 5.2.3, 5.2.4. Modules: M11. Interfaces: M11 records. Design: Fixed catalog. ADR: 13.

Existing verification: TC-5.2-1, TC-5.2-2, TC-5.2-3.

Design realization: M11 validates the fixed feature catalog and exactly one allowed status per profile-feature cell; it provides no weights or rankings.

Spec 5.3 Evidence and Cell Records

Clauses: 5.3.1, 5.3.2, 5.3.3, 5.3.4. Modules: M11. Interfaces: M11 records. Design: Evidence records. ADR: 13.

Existing verification: TC-5.3-1, TC-5.3-2, TC-5.3-3.

Design realization: M11 requires complete cell fields and resolvable source references or a gap reason; product-surface mismatches and source conflicts remain visible.

Spec 5.4 Review and Completion

Clauses: 5.4.1, 5.4.2, 5.4.3. Modules: M11. Interfaces: M11 review. Design: Independent review. ADR: 13.

Existing verification: TC-5.4-1, TC-5.4-2.

Design realization: M11 records independent reviewer identity and cell revision. Completion follows documentary evidence and review, not a live demonstration.

Spec 5.5 Scope Boundary and Presentation

Clauses: 5.5.1, 5.5.2, 5.5.3. Modules: M11. Interfaces: M11 export. Design: Separate assessment. ADR: 01 13.

Existing verification: TC-5.5-1, TC-5.5-2.

Design realization: M11 export states access dates and documentation-only scope; it rejects unsupported effectiveness claims and benchmark evidence as capability proof.

Traceability limitation: missing SRS NFR definitions and the undefined TC-4.5-3 reference require requirements reconciliation. Spec 4.5.3 is still covered by the timing assertion in TC-4.5-1; this does not create a missing test ID.

Appendix B Evidence and implementation status

The evidence below identifies the supplied baseline and preserved title-edit submissions. This revision did not rerun the application or certify current implementation behavior. Reported pilot outcomes come from the supplied DDD and earlier reviewed result records. New design contracts and diagrams do not change those outcomes.

Artifact	Immutable reference
Repository course baseline	9bd7f67aff60f6a2d701e62d7bdb95ed32424d60
Task 001 starting commit	be98d692807da26dac3c3a720d4d5cf157e9717d
Task 003 starting commit	4e258a96855d98edd10247f68c0b8265e53a8921
Title evaluator protocol	02996447e01bb284939d9b1a83c45ab033c22fb9
Antigravity title result	f02e023a62993171e22708818fdeb529fe12d1b1
Claude Code title result	5ae07ab6f5b53a14018559e8aac60a49da8b6ea8
Codex title result	1079551f892fb2d9e64239f03d0b4df5ce6f7ee5
Copilot title result	30c5294d7d4a36dc629d20bacebb6eb6d413b3e5

Area	Evidence status
Native task correctness	Reported external suites pass for title-edit; DELETE original/correction distinction retained.
C resource/budget controls	Target requirements; supplied DDD reports partial implementation.
Secret/license scans on pilots	Not established as completed; do not infer from API/browser success.
Common title-edit CI	Planned; requires actual workflow result before being presented as complete.
Schema 3 and release builder	Proposed design; approval and implementation pending.
Track A cells	Profile selection and source/reviewer records pending.

Original evidence should retain actual model labels, operating modes and timing methods. Avoid “default” unless recorded. A browser-version deviation or outside-workspace action is disclosed independently of application correctness. Any historical code observation needs a fresh pinned-code check before it is presented as current.

Appendix C AI disclosure and rubric coverage

Revision 1.5 was prepared with ChatGPT/Codex assistance on October 15, 2026. AI assistance covers the rewritten prose, new diagrams, interface/data contracts, ADR refinements, traceability extraction, consistency review and document formatting. These are proposed engineering decisions; team review and acceptance are not implied.

Content	Human responsibility before approval
Scope, comparison claims and evidence summary	Confirm intended research question, exact recorded configurations and pilot limitations.
Views and contracts in Sections 3–18	Review feasibility, isolation topology, lifecycle, schema changes and interface invariants.
Scoring and quantitative sections	Approve metric version and unresolved caps; confirm alignment with SRS V2.
ADRs, risks and public-impact analysis	Accept or revise trade-offs, policy assumptions, retention and stakeholder obligations.
Traceability and references	Confirm approved requirements baseline, clause coverage and linked evidence.
Historical disclosure	The supplied v1.4 PDF records prior team AI use. This revision does not retrospectively certify those review claims.

Rubric requirement	Coverage
Framework and architectural style	Sections 2–4; ADRs 01–14.
Logical, physical, data and behavioral views	Figures 1–8; Sections 4, 7–11.
Development and scenarios	Sections 9, 17–18.
Precise interfaces and data contracts	Sections 12–16.
Trade-offs, modularity and extensibility	Sections 5–6 and 22–24.
Security, observability, resilience and configuration	Sections 7, 10, 18 and 21.
Load, timing, capacity and validation	Sections 20 and 26–27.
Public health and broader impacts	Section 25.
Requirement traceability	Appendix A1–A5, plus reconciliation register.
AI notation, timesheet and presentation	This appendix; actual current timesheet and deck must accompany submission.

Timesheet entries must reflect actual contributor work and review time. No hours, reviewer names or approvals have been fabricated. Document completeness does not establish delivery of the in-class presentation or peer evaluation.

Appendix D References and glossary

[1] Microsoft 2 Team 3. Detailed Design Document, supplied PDF Detailed_Design_Doc_Microsoft#2 (3).pdf, revision history through 1.4, October 15, 2026. Historical source for this revision.

[2] Microsoft 2 Team 3. SRS working source, course_artifacts/sources/srs.md, repository commit 9bd7f67aff60f6a2d701e62d7bdb95ed32424d60. https://github.com/brunor560/Microsoft-Capstone

[3] Microsoft 2 Team 3. Agentic Developer Tool and Pipeline Feature Matrix Specification v0.2. course_artifacts/sources/feature_matrix_specification.md at the same repository commit.

[4] Microsoft 2 Team 3. Frozen task, evaluator and results branches. Exact title-edit commit references are listed in Appendix B. https://github.com/brunor560/Microsoft-Capstone/branches

[5] ISO/IEC/IEEE 42010:2022. Software, systems and enterprise — Architecture description. Framework referenced by the course rubric; no certification claim.

[6] P. Kruchten. The 4+1 View Model of Architecture. IEEE Software, 12(6), 42–50, 1995. View organization referenced by course materials.

[7] Docker. None network driver. https://docs.docker.com/engine/network/drivers/none/ Accessed October 15, 2026.

[8] GitHub. Secure use reference. https://docs.github.com/en/actions/reference/security/secure-use Accessed October 15, 2026.

[9] JSON Schema. Draft 2020-12. https://json-schema.org/draft/2020-12 Accessed October 15, 2026. Structural validation format; semantic checks remain application responsibilities.

[10] USF CIS 4910. Week 6 and Week 7 lecture materials; architecture_document_agentic_review_rubric.md in course_artifacts/references; professor rubric supplied October 15, 2026. Used for design coverage and submission obligations.

Term	Meaning
Harness	Software that connects a model/agent to tools, context, permissions and execution control.
Candidate	Preserved code produced by one attempt; not the baseline.
Frozen candidate	Snapshot captured after writers stop, identified by hashes.
Comparison group	Results sharing task/evaluator/environment/method conditions; configuration differences are explicit.
ADR	Architecture Decision Record: context, choice, alternatives and consequences.
NFR	Nonfunctional requirement; concrete targets must be traceable to an approved source.
SRS / DDD	Software Requirements Specification / Detailed Design Document.
CI / API / SHA	Continuous integration / application programming interface / secure hash algorithm.